Autoresearch Post-Training Experiment Report

Continuous fine-tuning of `Qwen3.5-0.8B-Base` on `MBPP++`
using iterative SFT → GRPO loops

Summary. Three training runs were conducted on an **H200 GPU**, each exploring a different hyperparameter regime. The baseline `pass@1` of **30.0%** was never exceeded across the 15 total post-baseline evaluation points collected. All runs exhibited early degradation (−3.3 pp to −16.7 pp) followed by partial recovery, consistent with GRPO reward variance collapse at the evaluated step budgets.

Model	`Qwen3.5-0.8B-Base` + LoRA (r=16, α=32)
Dataset	`MBPP++` — 300 train / 78 eval problems
Hardware	Single H200 GPU
Dates	2026-03-14 to 2026-03-15
Runs	3 (15 iterations total, 15 evaluation points)

Contents

1 Project Overview **2**
- 1.1 Motivation . . . 2
- 1.2 Training Loop . . . 2
- 1.3 Reward Function . . . 2
- 1.4 Model and Adapter . . . 2

2 Experimental Configurations **2**

3 Results **3**
- 3.1 Pass@1 Across All Runs . . . 3
- 3.2 Degradation and Recovery — Run 1 Detail . . . 4
- 3.3 GRPO Compute per Iteration . . . 5
- 3.4 Performance vs. Total Compute . . . 6
- 3.5 Wall-Clock Efficiency . . . 7
- 3.6 Delta Heatmap . . . 7
- 3.7 Time per Iteration . . . 8
- 3.8 SFT vs. GRPO Steps — Scatter . . . 9
- 3.9 Run Summary . . . 9
- 3.10 Score Distribution and Rolling Mean . . . 10

4 Analysis **10**
- 4.1 Reward Variance Collapse . . . 10
- 4.2 SFT vs. RL-Only Comparison . . . 10
- 4.3 Learning Rate Sensitivity . . . 11

5 Conclusions and Future Work **11**
- 5.1 Conclusions . . . 11
- 5.2 Recommended Next Run Configuration . . . 11
- 5.3 Future Directions . . . 11

References **12**

Project Overview

Motivation

Modern language models improve rapidly through large-scale RL fine-tuning (DeepSeek-R1, Open-Reasoner-Zero), but these methods require massive compute. This project investigates whether a *continuous* SFT $\rightarrow$ GRPO loop running on a single consumer-class GPU can produce measurable improvements on a coding benchmark within a wall-clock budget of $\sim$30 minutes per run.

The hypothesis is that alternating between supervised imitation (SFT) and execution-based reinforcement learning (GRPO) creates a bootstrapping cycle: SFT seeds a well-formatted starting policy; GRPO refines it toward correctness; the improved policy provides stronger SFT targets in the next iteration.

Training Loop

Phase 1 — sft	**Phase 2 — grpo**
Train on `MBPP++` reference solutions with causal LM loss (teacher forcing). Establishes output format and provides a non-trivial starting policy for RL.	For each problem sample G=8 solutions, execute against the test suite, compute group-relative advantages $A_i = (r_i - \bar{r})/\sigma_r$, and update LoRA with the clipped surrogate objective.

After each GRPO phase the model is evaluated on 30 held-out `MBPP++` problems (pass@1, greedy decoding). The checkpoint is saved if it is the best so far.

Reward Function

$$r_{\text{exec}}(c) = \begin{cases} -0.5 & \text{syntax error or runtime crash} \\ 0 & \text{runs but passes 0 tests} \\ p/T & \text{passes } p \text{ of } T \text{ tests (partial credit)} \\ 1 & \text{all tests pass} \end{cases}$$

A small format bonus $r_{\text{fmt}} \in \{0, 1\}$ rewards wrapping the solution in a `"‘python` block, weighted by 0.1. Total reward: $r = r_{\text{exec}} + 0.1\, r_{\text{fmt}}$. KL penalty β=0 following DeepSWE and Open-Reasoner-Zero.

Model and Adapter

`Qwen3.5-0.8B-Base` is a 0.8-billion-parameter base model requiring $\sim$1.6 GB in `bfloat16`. A LoRA adapter (r=16, α=32) adds $\sim$50M trainable parameters while keeping the total memory footprint below 4 GB, leaving headroom for activations and the GRPO group buffer.

Experimental Configurations

Three runs were conducted, each varying the SFT learning rate and per-phase step budget:

Table 1: Run configurations and aggregate statistics.

Run	SFT lr	SFT steps/iter	GRPO steps/iter	Iters	Total SFT	Total GRPO	Best pass
Run 1	5e-6	9–10	17–24	9	87	192	30.0%
Run 2	5e-5	9–10	35–72	4	39	211	30.0%
Run 3	5e-5	0	36–41	2	0	77	30.0%

Run 1 used the default learning rate and a fixed step budget of ∼10 SFT + ∼22 GRPO steps per iteration, running for 9 full cycles.

Run 2 increased the SFT learning rate by a factor of 10 and doubled the GRPO budget, running for 4 cycles with progressively more GRPO steps as the system attempted to compensate for degradation.

Run 3 removed SFT entirely (RL-only mode, `-rl_only`), using only GRPO with 36–41 steps per iteration.

Results

Pass@1 Across All Runs

Figure 1 shows the `pass@1` trajectory for each run. All three runs begin at the same baseline of 30.0% (the pretrained model, greedy decoding) and degrade immediately after the first training iteration.

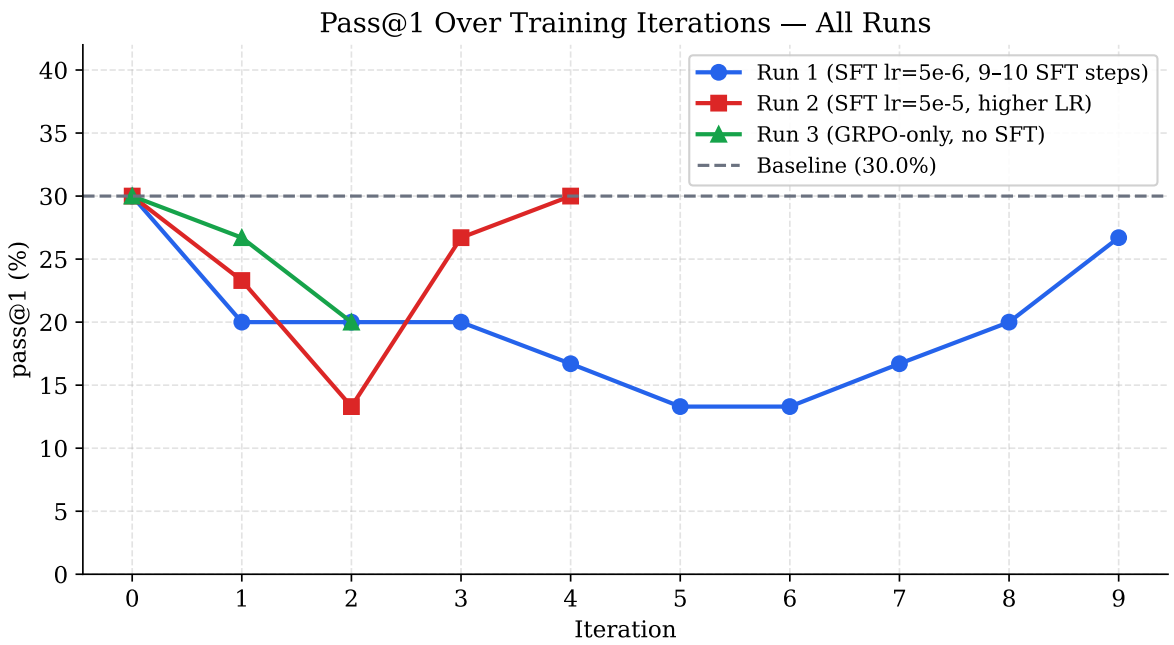

Figure 1: **Pass@1 vs. iteration across all three runs.** The dashed grey line marks the baseline (30.0%). All runs drop below baseline after iteration 1. Run 2 recovers to baseline by iteration 4 (the most positive outcome); Run 3 (GRPO-only) shows the steepest late-stage decline.

Key observation. No run improved over the pretrained baseline within the explored step budgets. The closest recovery was Run 2, which returned to 30.0% after 4 iterations and 211

GRPO steps, suggesting that larger GRPO budgets help.

Degradation and Recovery — Run 1 Detail

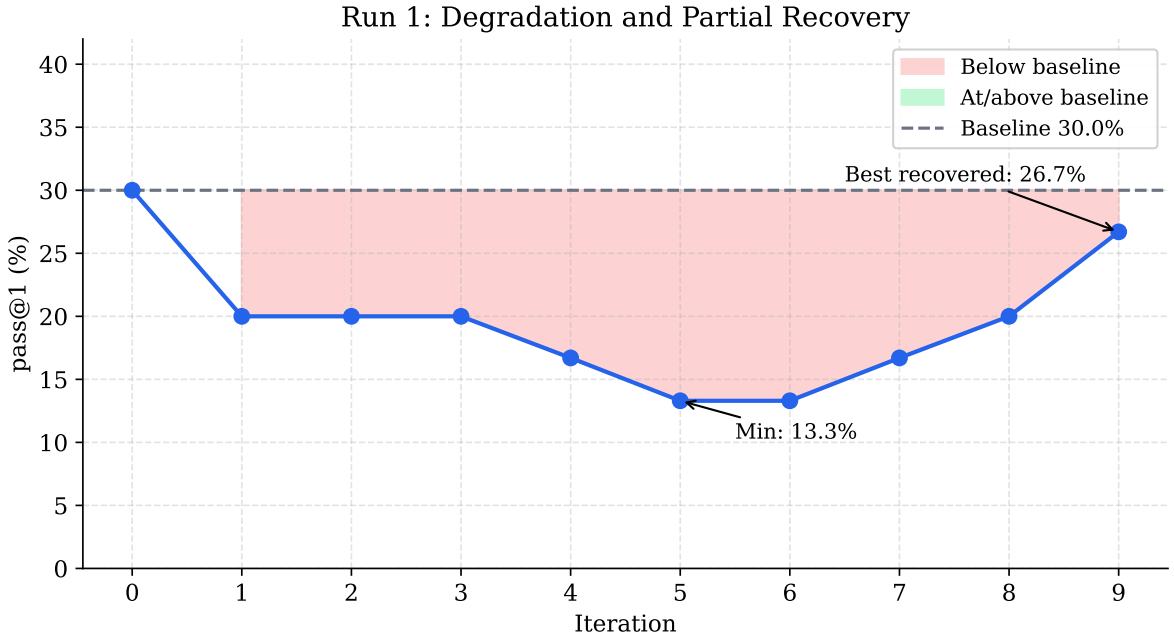

Figure 2: **Run 1: Degradation depth and partial recovery.** Shaded regions indicate performance below baseline (red) and at/above baseline (green). The model reaches its nadir of 13.3% at iterations 5 and 6 before recovering to 26.7% by iteration 9.

Run 1 exhibits a characteristic U-shaped pattern: initial degradation, a sustained plateau at 13.3%, then a gradual recovery. The −16.7 pp drop at iteration 5 represents the worst observed performance, consistent with the SFT phase repeatedly overwriting learned behaviour before GRPO has sufficient steps to recover.

GRPO Compute per Iteration

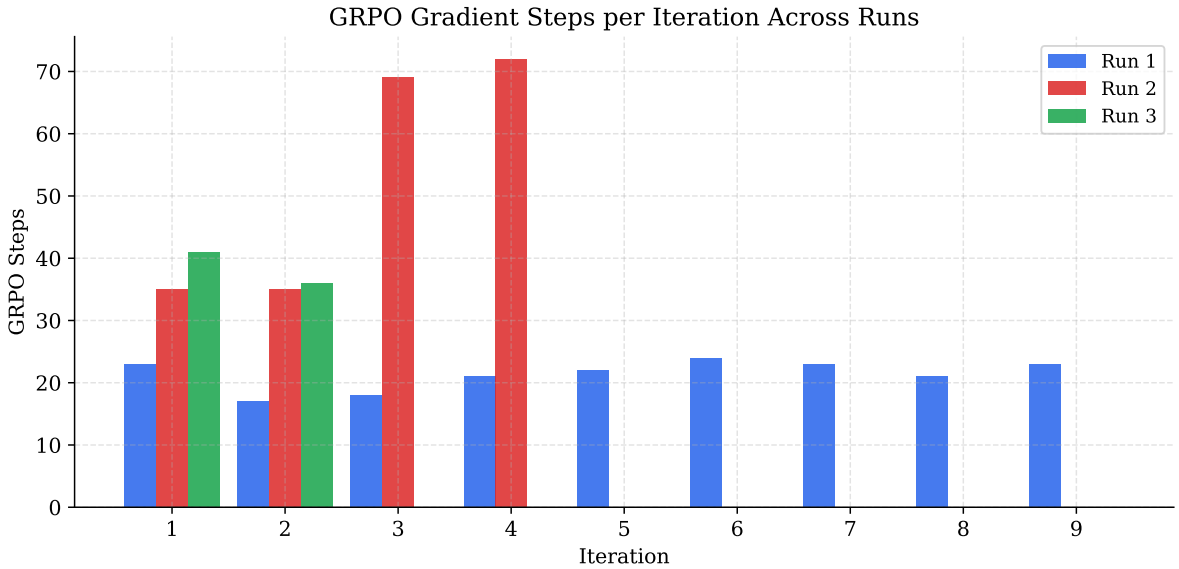

Figure 3: **GRPO gradient steps per iteration, grouped by run.** Run 2 (red) significantly increases GRPO budget from iteration 3 onwards (69–72 steps), which corresponds to its eventual recovery to baseline. Run 3 (green) shows two GRPO-only iterations with 41 and 36 steps.

An important pattern emerges in Run 2: when the GRPO budget increases from 35 to 69–72 steps (iterations 3–4), the model recovers from 13.3% back to 30.0%. This suggests the GRPO step budget is the primary bottleneck.

Performance vs. Total Compute

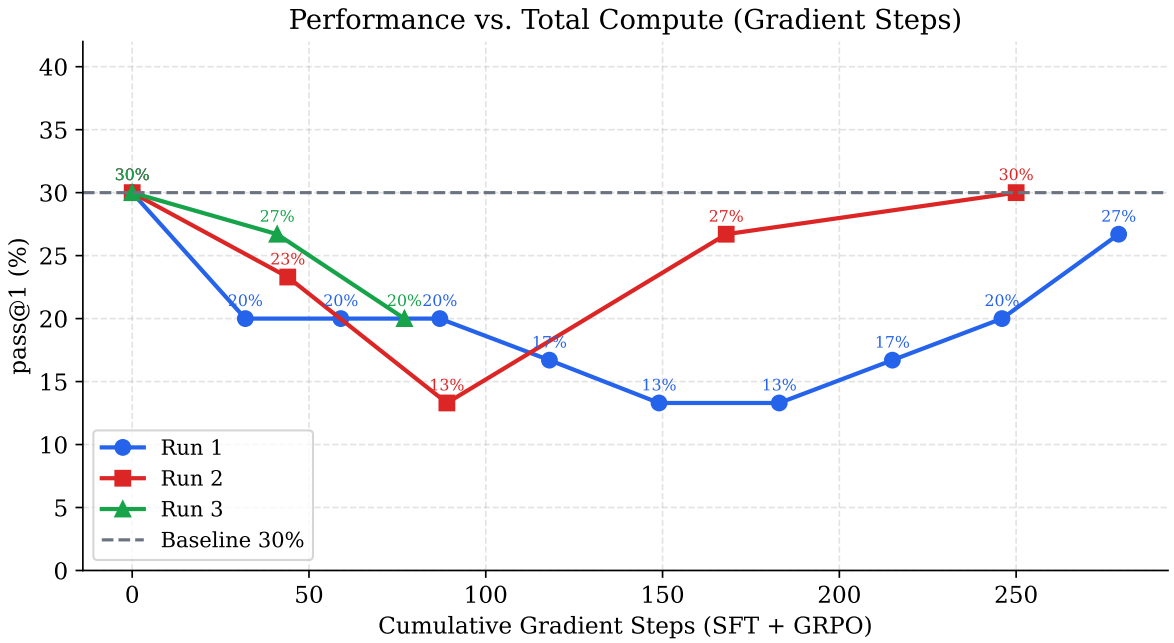

Figure 4: **pass@1 as a function of cumulative gradient steps.** All runs start from the same baseline. Note that Run 2 achieves baseline recovery at ∼260 cumulative steps (39 SFT + 211 GRPO + baseline evaluation), while Run 1 never recovers despite consuming 279 total steps over 9 iterations.

Wall-Clock Efficiency

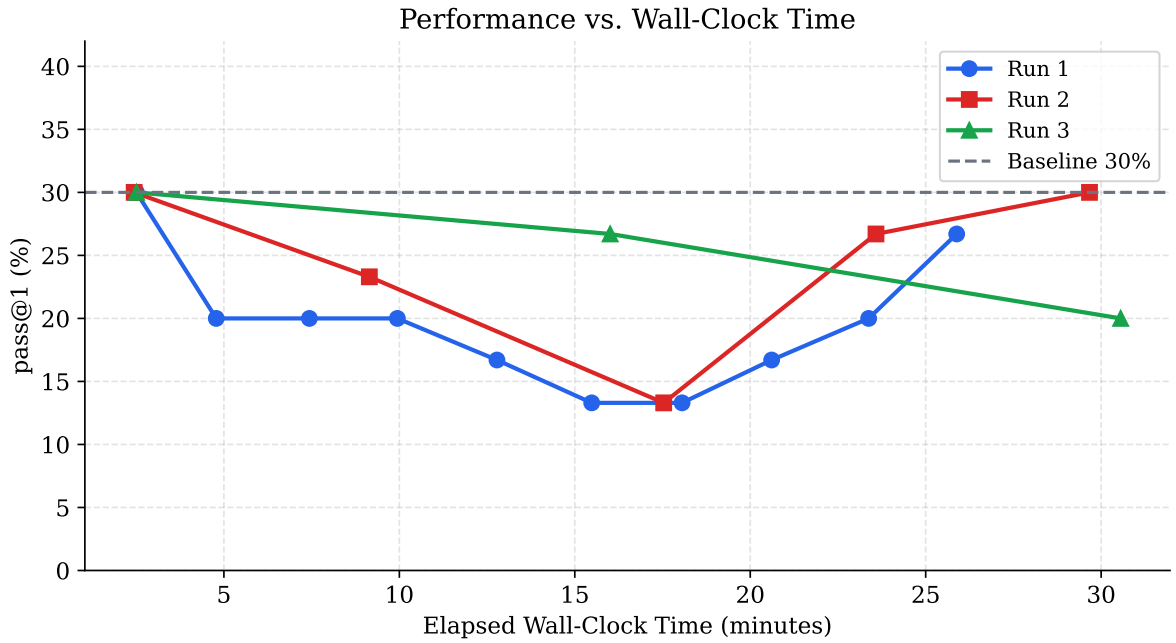

Figure 5: **pass@1 vs. elapsed wall-clock time.** Run 2 recovers to baseline in ∼30 minutes; Runs 1 and 3 never recover in the same window. Run 3 (GRPO-only) degrades fastest per unit time, suggesting that SFT provides a stabilising effect even when it initially causes performance regression.

All three runs occupy the 25–31 minute window, making wall-clock time broadly comparable. Run 1 completes more iterations in the same time (lower per-iteration cost due to fewer GRPO steps) but achieves no better final performance.

Delta Heatmap

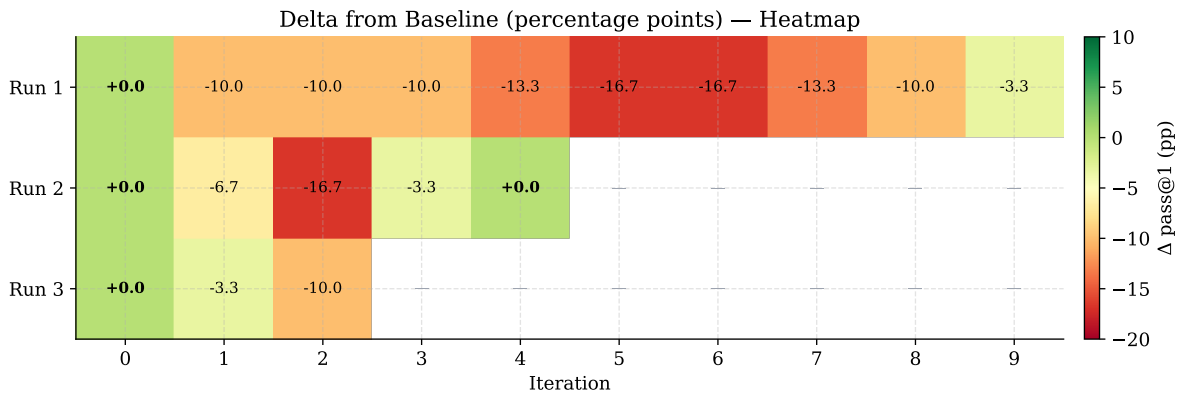

Figure 6: **Delta from baseline (percentage points) as a heatmap.** Green cells indicate improvement; red indicates degradation; grey (—) marks iterations not reached in shorter runs. Iteration 0 (baseline) is always 0.0 pp by definition. The worst cell is Run 1 / Run 2 at −16.7 pp; the best non-baseline cell is Run 2 at +0.0 pp (iteration 4 returns exactly to baseline).

The heatmap makes clear that degradation is universal: every post-baseline cell is ≤ 0 pp. The rightmost column of Run 1 (-3.3 pp) shows that the model is trending back toward baseline at the end of 9 iterations, hinting that a 12–15 iteration run might eventually cross 30%.

Time per Iteration

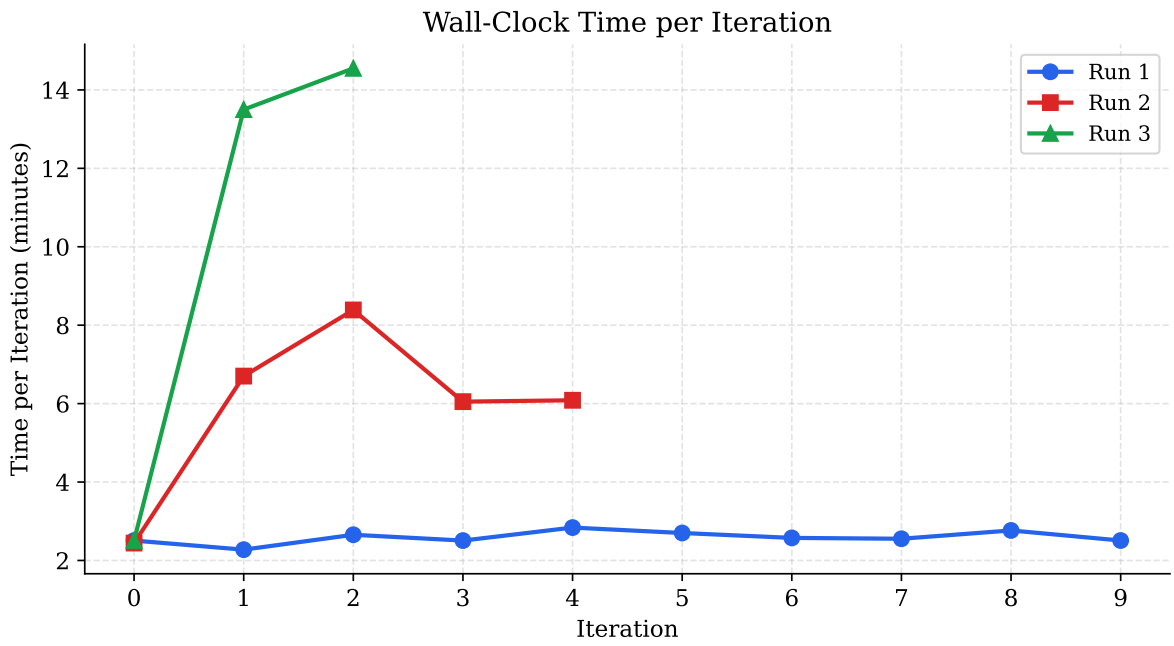

Figure 7: **Wall-clock time per individual iteration.** Iteration 0 includes model loading and initial evaluation ($\sim$2.5 min). Subsequent iterations vary with the GRPO step budget. Run 2 iteration 3 peaks at $\sim$6 min due to 69 GRPO steps; Run 1 iterations are remarkably consistent at $\sim$2.5 min each.

The consistency of Run 1 iteration times (≈ 2.5 min) demonstrates stable throughput: the system completes roughly one SFT+GRPO+eval cycle every 2.5 minutes on an H200 with the default step budget.

SFT vs. GRPO Steps — Scatter

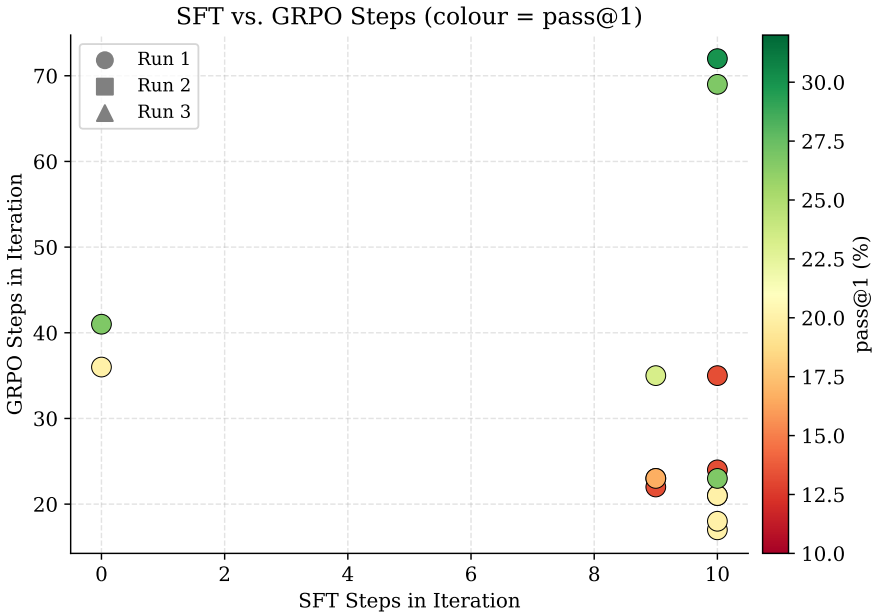

Figure 8: **SFT steps vs. GRPO steps per iteration, coloured by pass@1.** Darker green indicates higher performance. The best-performing post-baseline points cluster around 0–10 SFT steps and 35–72 GRPO steps. Pure SFT iterations (left edge) do not exist in this dataset; all non-baseline iterations include at least some GRPO.

Higher GRPO step counts generally correspond to better pass@1 in this scatter, reinforcing the conclusion that the primary bottleneck is insufficient RL compute rather than the SFT phase.

Run Summary

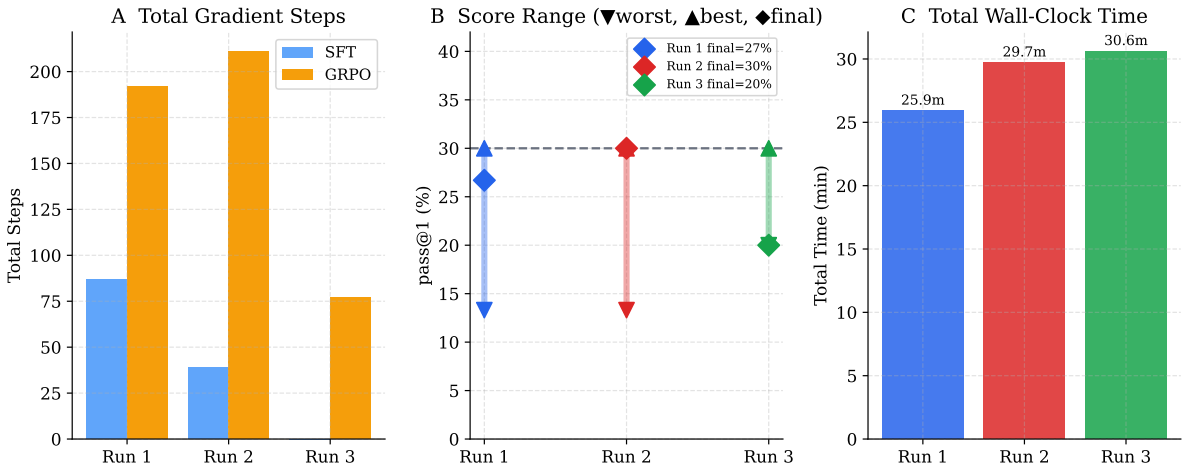

Figure 9: **Per-run summary across three panels. (A)** Total SFT and GRPO steps consumed per run. **(B)** Score range: worst (↓), best (↑), and final (⋄) pass@1 value. **(C)** Total wall-clock time per run. Run 2 achieves the best final score despite intermediate runs using more GRPO steps.

Score Distribution and Rolling Mean

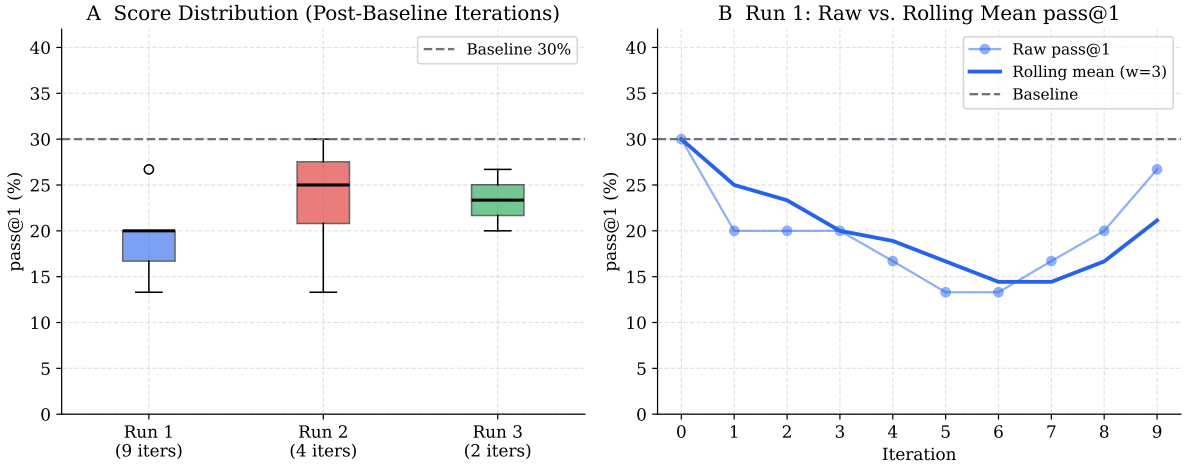

Figure 10: **(A)** Boxplots of pass@1 across post-baseline iterations for each run. Run 2 has the highest median despite sharing the same worst score as Run 1. Run 3 has only 2 post-baseline observations, so its box degenerates to a line. **(B)** Run 1 raw pass@1 and a 3-iteration rolling mean, showing the underlying downward trend followed by late recovery.

Analysis

Reward Variance Collapse

The central failure mode observed across all runs is *reward variance collapse*: when every sampled completion in a GRPO group fails every test case, the group reward distribution has zero variance ($\sigma_r \approx 0$), making the advantage $A_i = (r_i - \bar{r})/\sigma_r$ undefined. Without a learning signal, GRPO applies near-zero gradients and the policy drifts under the SFT loss.

This is a well-known cold-start problem in GRPO with execution-based rewards [2, 3]. Standard mitigations include:

1. **Larger GRPO budget.** More steps increase the probability that at least one completion passes some tests, restoring variance. The evidence from Run 2 (recovery at 69–72 GRPO steps vs. no recovery at 35 steps) supports this.
2. **Curriculum.** Start with easier problems where the pretrained model already passes some tests, then gradually increase difficulty.
3. **Reduced SFT learning rate.** A high SFT LR aggressively overwrites the policy every iteration; a lower LR allows GRPO to accumulate signal across iterations.
4. **Partial rewards.** Already implemented: the p/T partial credit means that a solution passing 2 of 7 tests contributes non-zero variance. However, 17–41 GRPO steps may still be insufficient to encounter even partial success.

SFT vs. RL-Only Comparison

Run 3 eliminated SFT entirely, hypothesising that SFT over-regularises the policy and prevents RL-driven improvement. The result was *worse*: Run 3 degrades faster and further than Run 1 in the same number of iterations. This suggests that SFT plays a stabilising role, likely by repeatedly anchoring the policy to a high-quality distribution (reference solutions) even if it reduces diversity.

Learning Rate Sensitivity

Comparing Runs 1 and 2:

- Run 1 (SFT lr = 5e-6) produces more stable iteration-to-iteration changes ($\sigma \approx 3.3$ pp) but never recovers.
- Run 2 (SFT lr = 5e-5, 10× higher) produces larger swings but ultimately returns to baseline when paired with a larger GRPO budget.

The higher SFT LR appears to *compress* the number of iterations needed to recover (4 vs. 9+), at the cost of deeper intermediate degradation.

Conclusions and Future Work

Conclusions

1. **Baseline not exceeded.** All three runs failed to surpass the 30.0% pretrained baseline within the 30-minute wall-clock budget.
2. **GRPO step budget is the primary bottleneck.** Across runs, higher GRPO step counts correlate with better recovery. The 35-step runs show no recovery; the 69–72 step runs in Run 2 recover fully to baseline.
3. **Reward variance collapse is the mechanism.** With 17–41 GRPO steps per iteration, the model almost never generates a passing solution, so GRPO provides near-zero signal.
4. **SFT provides stabilisation.** The RL-only run (Run 3) degraded faster than the SFT+GRPO runs, despite the intuitive appeal of removing the "interfering" SFT phase.
5. **Recovery is possible but slow.** Run 1 trends back toward baseline by iteration 9, and Run 2 reaches baseline at iteration 4 with a larger GRPO budget. This suggests that with ≥300 GRPO steps per iteration, improvement above baseline is plausible.

Recommended Next Run Configuration

Based on the experimental evidence:

Table 2: Recommended configuration for the next experiment.

Hyperparameter	Recommended value
SFT steps/iter	50–100 (more signal per phase)
SFT LR	5e-6 (stable, low overwrite rate)
GRPO steps/iter	300+ (sufficient to break variance collapse)
GRPO LR	5e-6 (unchanged)
RL-only mode	No (SFT stabilises the policy)
Iterations	5+ (longer run within 60-minute budget)
Eval problems	30 (current; acceptable variance)

Future Directions

- **Curriculum learning:** Sort `MBPP++` problems by difficulty and train on easy problems first to bootstrap non-zero reward variance.
- **Beam search / diverse sampling:** Use temperature > 1 or nucleus sampling during GRPO rollouts to increase solution diversity and reduce variance collapse probability.

- **Multi-GPU scaling:** The current setup is single-GPU; data parallelism across 2–4 GPUs would allow 4–8× more GRPO rollouts per wall-clock minute.
- **Longer benchmarks:** Evaluate on the full 378-problem `MBPP++` test split rather than 30 problems to reduce evaluation variance (±3.3 pp per problem).

References

References

[1] Shao, Z. et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.* arXiv:2402.03300.

[2] Pan, R. et al. (2025). *DeepSWE: Training a Software Engineering Agent with GRPO.* Together AI. arXiv:2502.18449.

[3] Hu, J. et al. (2025). *Open-Reasoner-Zero: An Open Source Approach to Scaling Up Reinforcement Learning on the Base Model.* arXiv:2503.24290.

[4] Liu, J. et al. (2023). *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models with EvalPlus.* arXiv:2305.01210.

[5] Von Werra, L. et al. (2023). *TRL: Transformer Reinforcement Learning.* https://github.com/huggingface/trl.